



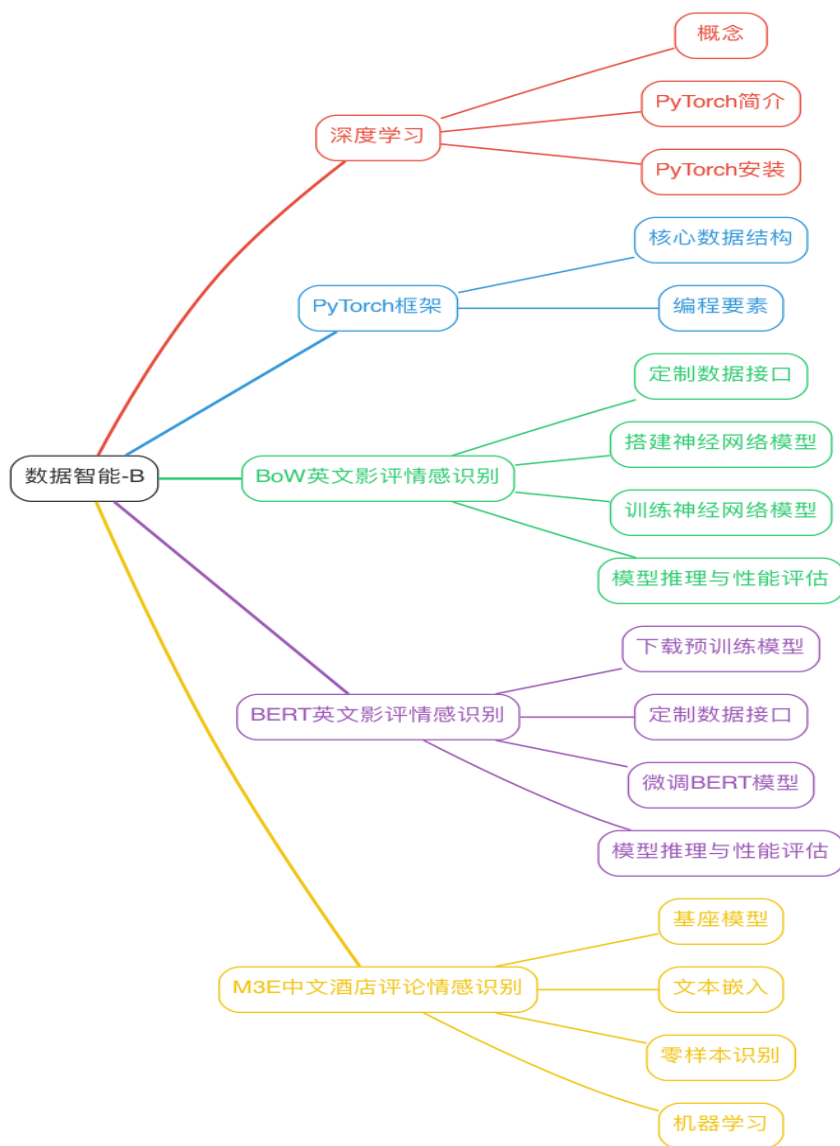
# 《实用Python编程》

## 第10讲：数据智能-B

实用Python编程 教研组



# 提纲



# 深度学习 (deep learning)

- 一种新的机器学习范式
  - 以多层人工神经网络为架构，以原始数据为输入，以目标任务为输出，具备端到端学习能力 (end-to-end learning)
  - 将“特征提取”与“分类器学习”融为一体，相互促进

传统机器学习“特征提取”与“分类器学习”之间不存在反馈机制

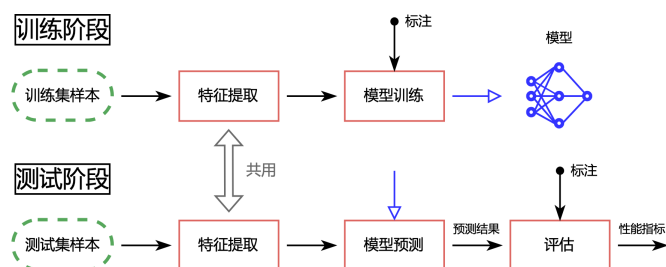
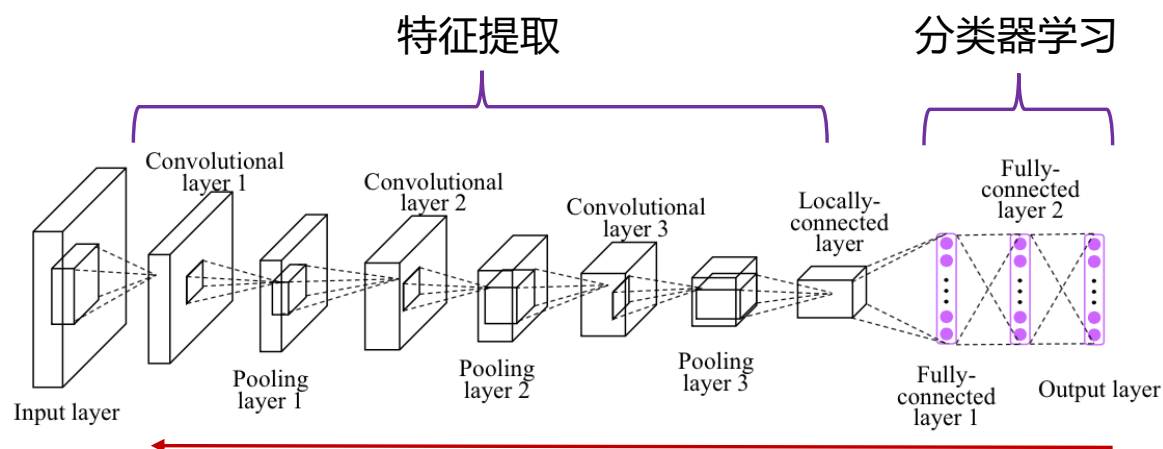


图 5.2: 机器学习典型的两阶段流程

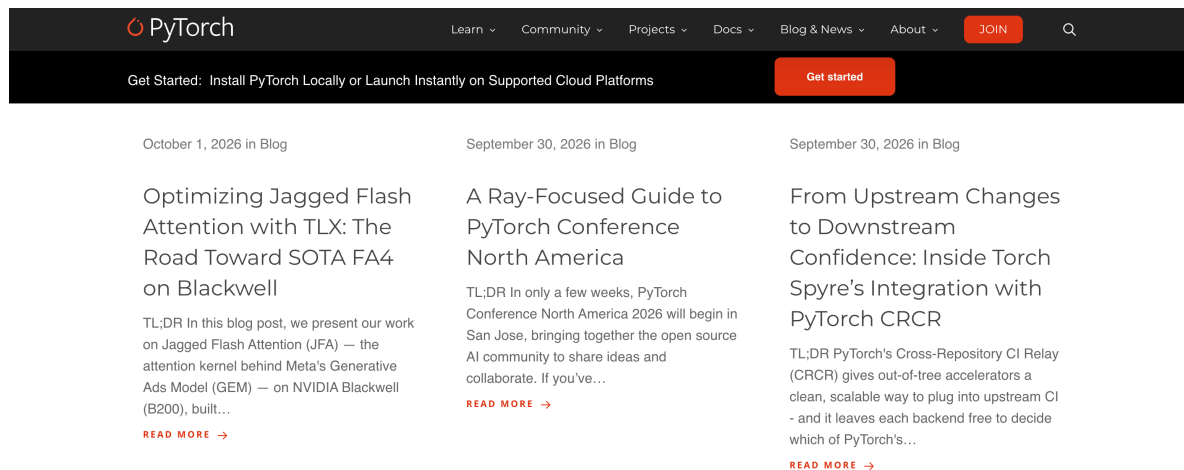


分类错误经反向传播反馈给特征提取层



# 从概念到实践

- 利用PyTorch进行深度学习
  - PyTorch：目前最流行的深度学习框架
  - 以张量（Tensor）为核心数据结构，实现带自动求导功能的深度神经网络
  - 官网：<https://pytorch.org/get-started/locally/>



# PyTorch安装

- 参考官网导引，按操作系统、计算装置等选择，生成安装命令

|                   |                                |           |                   |          |
|-------------------|--------------------------------|-----------|-------------------|----------|
| PyTorch Build     | Stable (2.8.0)                 |           | Preview (Nightly) |          |
| Your OS           | Linux                          | Mac       | Windows           |          |
| Package           | Pip                            | LibTorch  | Source            |          |
| Language          | Python                         |           | C++ / Java        |          |
| Compute Platform  | CUDA 12.6                      | CUDA 12.8 | CUDA 12.9         | ROCm 6.4 |
|                   |                                |           |                   | Default  |
| Run this Command: | pip3 install torch torchvision |           |                   |          |

- 检查PyTorch是否已就绪、是否有GPU可用

```
1 import torch
2 print(torch.__version__)
3 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
4 print(device)
```

2.10.0  
cpu

没有配备GPU的笔记本电脑  
也能开展深度学习任务，  
只是慢一些



# 深度学习环境配置

- 除了PyTorch，本课件案例还涉及下列第三方库

| 库名                    | 主要用途               | 安装命令                                           |
|-----------------------|--------------------|------------------------------------------------|
| huggingface-hub       | 从Hugging Face下载模型  | <code>pip install huggingface_hub</code>       |
| sentence-transformers | 加载基座模型             | <code>pip install sentence-transformers</code> |
| tqdm                  | 打印进度条              | <code>pip install tqdm</code>                  |
| transformers          | 加载Transformers系列模型 | <code>pip install transformers</code>          |



# PyTorch框架

- 核心数据结构
  - Tensor: 样本、标签、权重、特征、梯度等均为Tensor对象
- 编程要素
  - Dataset / DataLoader: 将原始数据变为模型输入
  - Module: 构造模型 (自定义神经网络结构)
  - CrossEntropyLoss: 用于分类的交叉熵损失
  - AdamW: 常用优化器, 指导模型如何调整自身参数



# 认识Tensor

- 简单理解，Tensor就是深度学习版的ndarray
  - 与ndarray的接口高度相似
  - 但支持自动微分、GPU加速、混合精度等



**GPU 加速**

海量并行计算



**自动微分**

梯度自动求解



**混合精度**

省显存提速度

- 与ndarray可互转

```
t = torch.from_numpy(arr)
```

```
arr = t.numpy()
```

# 创建Tensor对象

- 常用方式

- 按指定形状
- 复制已有列表
- 复制已有ndarray
- 复制其他Tensor

```
1 arr = np.array([1, 2, 3])
2 t = torch.from_numpy(arr)
3 arr[0] = 9
4 print(arr, t)
5 t[0] = 1
6 print(arr, t)
```

```
[9 2 3] tensor([9, 2, 3])
[1 2 3] tensor([1, 2, 3])
```

```
1 import torch
2 import numpy as np
3 # 使用随机值或常量值创建指定形状的张量
4 shape = (2, 3)
5 zeros_tensor = torch.zeros(shape)
6 ones_tensor = torch.ones(shape)
7 rand_tensor = torch.rand(shape)
8 # 从Python列表创建
9 data = [[1, 2], [3, 4]]
10 x_data = torch.tensor(data)
11 # 从NumPy数组创建 (共享底层内存)
12 np_array = np.array(data)
13 x_np = torch.from_numpy(np_array)
14 # 基于现有Tensor对象创建, 将继承输入对象的形状、数据类型等属性
15 x_ones = torch.ones_like(x_data) # 创建一个2x2的元素值均为1的Tensor
16 x_rand = torch.rand_like(x_data, dtype=torch.float) # 创建一个2x2的元素为随机浮点数的Tensor
```

⚠ from\_numpy 与原数组共享内存, 改一方另一方也变



# 查看Tensor对象的属性

- 常用属性

- ndim: 维数
- shape: 以元组形式返还Tensor对象的形状，即每一维的长度
- dtype: 元素数据类型
- device: 承担当前对象计算任务的算力资源（默认为CPU）
- requires\_grad: 是否为该对象计算梯度（默认为False）

```
1 x = torch.rand((3, 4))
2 print(x)
3 print(x.ndim, x.shape, x.dtype, x.device, x.requires_grad)
```

```
tensor([[0.5217, 0.6527, 0.0611, 0.6350],
        [0.9570, 0.6523, 0.2518, 0.6213],
        [0.7632, 0.6986, 0.5673, 0.2776]])
2 torch.Size([3, 4]) torch.float32 cpu False
```

创建一个3行4列的随机矩阵

- 维度为2
- 元素为浮点数
- 默认放在CPU上计算
- 不计算梯度



# Tensor运算

- 简单算术运算

```
1 tensor = torch.ones((2, 2))
2 print(tensor)
3 print(tensor + 2) # 张量与标量相加
4 print(tensor - 1) # 张量与标量相减
5 print(tensor * 3) # 张量与标量相乘
6 print(tensor / 2) # 张量与标量相除
```

```
tensor([[1., 1.],
        [1., 1.]])
tensor([[3., 3.],
        [3., 3.]])
tensor([[0., 0.],
        [0., 0.]])
tensor([[3., 3.],
        [3., 3.]])
tensor([[0.5000, 0.5000],
        [0.5000, 0.5000]])
```

标量先自行数据广播 (broadcasting) , 即参考当前对象的尺寸自动复制, 产生一个同尺寸的Tensor



# Tensor运算

- 逐元素相乘 (\*) vs 矩阵乘法 (@)

```
1 a = torch.tensor([[1, 0], [0, 1]])
2 b = torch.tensor([[1, 2], [3, 4]])
3 c = a * b # 等同于 torch.mul(a, b)
4 d = a @ b # 等同于 torch.matmul(a, b)
5 print(c)
6 print(d)
```

```
tensor([[1, 0],
        [0, 4]])
tensor([[1, 2],
        [3, 4]])
```

注意a是一个单位矩阵，也可用`torch.eye(2)`产生



# Tensor运算

- 形变 (reshape)
  - 若某一维长度值置为-1, 则交由程序自动推断该维的长度

```
1 a = torch.rand((4, 4))
2 b = a.reshape(-1, 8)
3 c = a.reshape(2, -1)
4 d = a.reshape(-1)
5 print(a.shape, b.shape, c.shape, d.shape)
```

这四个Tensor对象的维数分别是多少？

- 回想一下, 维数对应的属性是？

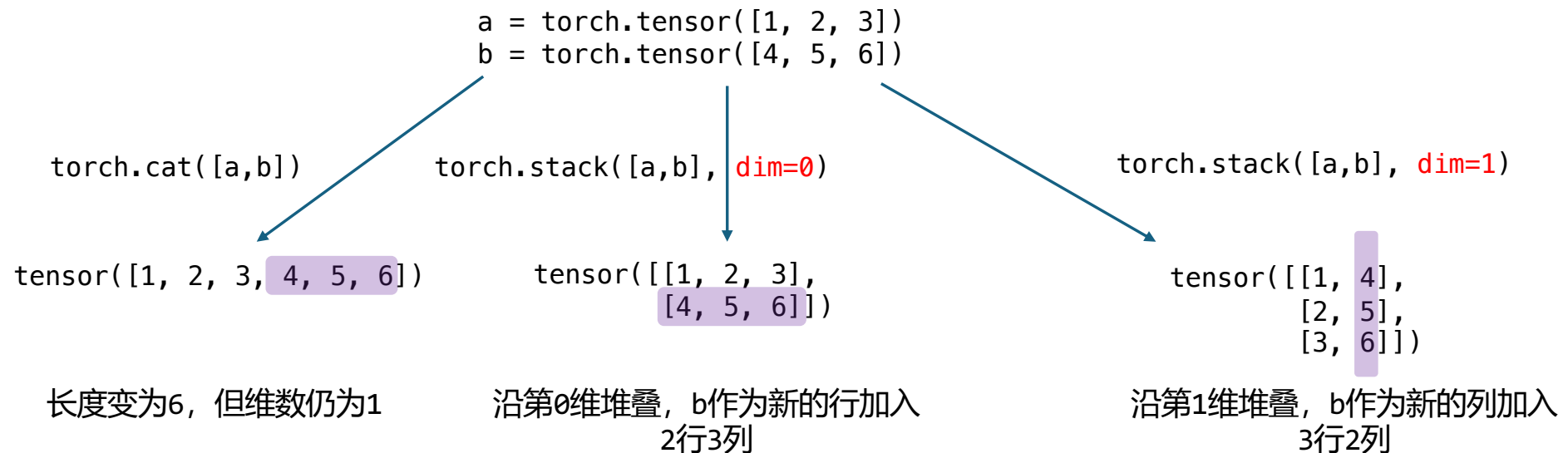
```
torch.Size([4, 4]) torch.Size([2, 8]) torch.Size([2, 8]) torch.Size([16])
```

|   |                                                                                                                                                           |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| a | tensor([[0.3186, 0.2571, 0.3559, 0.8078],<br>[0.4759, 0.7674, 0.0303, 0.2203],<br>[0.9144, 0.9560, 0.1701, 0.1486],<br>[0.5200, 0.9314, 0.4593, 0.3462]]) |
| b | tensor([[0.3186, 0.2571, 0.3559, 0.8078, 0.4759, 0.7674, 0.0303, 0.2203],<br>[0.9144, 0.9560, 0.1701, 0.1486, 0.5200, 0.9314, 0.4593, 0.3462]])           |
| c | tensor([[0.3186, 0.2571, 0.3559, 0.8078, 0.4759, 0.7674, 0.0303, 0.2203],<br>[0.9144, 0.9560, 0.1701, 0.1486, 0.5200, 0.9314, 0.4593, 0.3462]])           |
| d | tensor([0.3186, 0.2571, 0.3559, 0.8078, 0.4759, 0.7674, 0.0303, 0.2203, 0.9144,<br>0.9560, 0.1701, 0.1486, 0.5200, 0.9314, 0.4593, 0.3462])               |

# Tensor运算

- 合并两个Tensor对象

- 拼接 (`torch.cat`) : 不增加维数
- 堆叠 (`torch.stack`) : 维数加一, 即增加一个新的数据维度



# Tensor运算

- 自动求导

- Tensor对象的requires\_grad属性需要设置为True
- 调用`y.backward()`自动计算梯度，结果存放在`x.grad`

```
1 x = torch.tensor(2.0, requires_grad=True)
2 y = x**2 + 3*x + 1
3 y.backward() # 自动计算梯度
4 print(x.grad)
```

tensor(7.)

目标函数:  $y=x^2+3x+1$

手动求导:  $dy/dx=2x+3$

⚡ 我们只需要搭建好网络架构，明确目标函数，梯度计算可完全交给Tensor来完成

⚠ 目标函数输出值必须为标量才能直接调用`backward()`



# Tensor运算

- 使用to方法把Tensor移到特定计算装置 (device) 上计算

```
1 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2 x = torch.rand((2, 2)).to(device) # 移动到GPU或CPU上来完成后续计算
3 print(x.device)
```

cpu

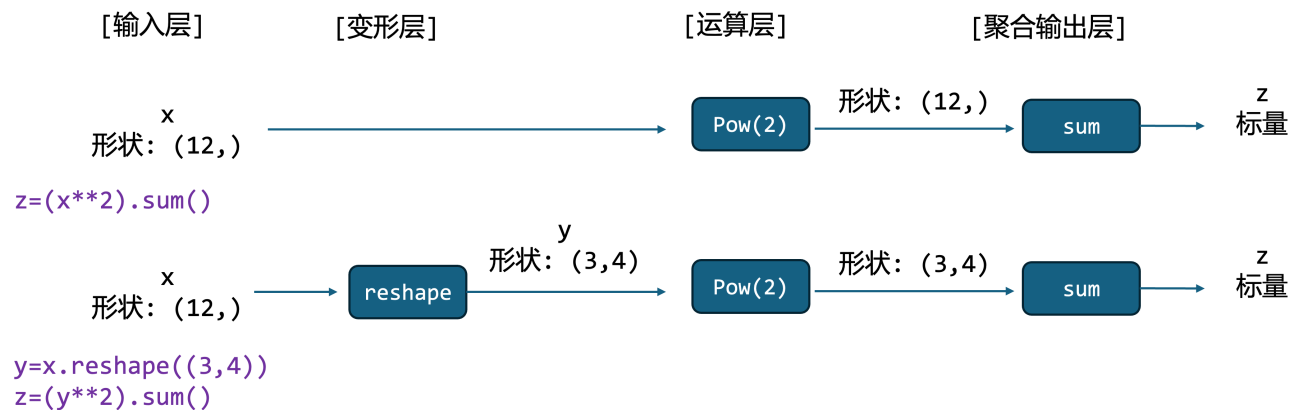
⚠ 参与运算的模型和数据都必须位于同一device, 否则报 RuntimeError

## 课堂练习-5-2-1

给定序列 $x=[0,1,2,\dots,11]$

- 将 $x$ 形变为一个 $3 \times 4$ 的Tensor对象 $y$
- 分别计算 $y$ 每一行、每一列的均值
- 对于函数 $z=(x**2).sum()$ ，计算 $x$ 的梯度
- 【延伸】若将目标函数定义为  $z=(y**2).sum()$ ，如何获得 $y$ 的梯度？

```
tensor([[ 0.,  1.,  2.,  3.],  
        [ 4.,  5.,  6.,  7.],  
        [ 8.,  9., 10., 11.]])
```





# PyTorch编程要素

- 核心任务是如何利用标注数据，让网络权重（模型参数）往损失函数最小化的方向调整
  - 具体调整策略由优化器根据损失函数自动求导产生的梯度决定
- 需要在PyTorch框架下回答四个问题
  - 如何加载（标注）数据？
  - 如何构造（网络）模型？
  - 如何指定损失函数？
  - 如何指定优化器？



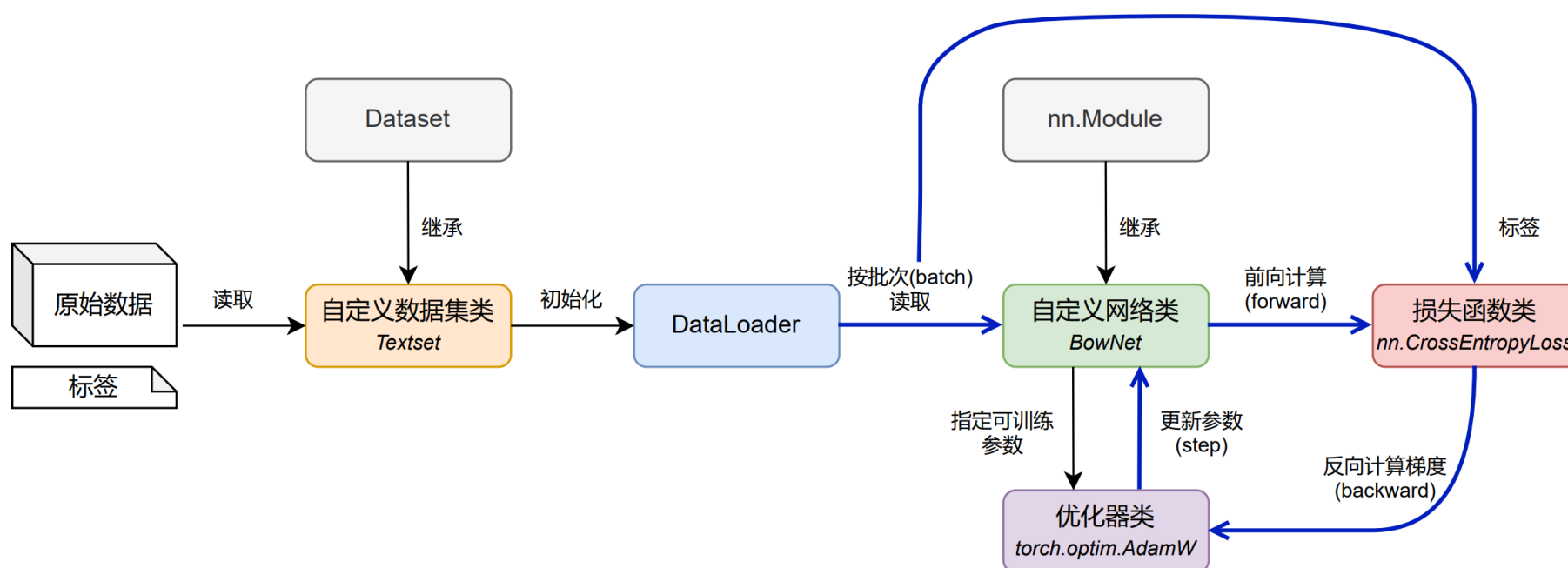
# PyTorch编程要素

## • 问题与torch类的对应关系

|           | 类                             | 所属模块                          | 作用                                                 |
|-----------|-------------------------------|-------------------------------|----------------------------------------------------|
| 如何加载数据?   | <code>Dataset</code>          | <code>torch.utils.data</code> | 通过继承 <code>Dataset</code> 来定义如何取一条样本               |
|           | <code>DataLoader</code>       | <code>torch.utils.data</code> | 将指定的 <code>Dataset</code> 对象封装成可迭代对象按批次送入模型进行训练或推理 |
| 如何构造模型?   | <code>Module</code>           | <code>torch.nn</code>         | 通过继承 <code>Module</code> 来定义神经网络结构                 |
| 如何指定损失函数? | <code>CrossEntropyLoss</code> | <code>torch.nn</code>         | 用于分类的交叉熵损失                                         |
|           | <code>MSELoss</code>          | <code>torch.nn</code>         | 用于回归的均方误差损失                                        |
| 如何指定优化器?  | <code>AdamW</code>            | <code>torch.optim</code>      | 常用优化器                                              |

# PyTorch编程要素

- 各核心类对象在训练过程中的角色及交互关系





# 案例1：基于BoW的英文影评情感识别

- 沿用5.2.1节数据，复用BoW特征提取器
- 熟悉PyTorch深度学习 workflow





# Step 1. 定制数据接口

- 数据接口用于将N个训练样本分批次“喂给”模型进行学习
  - 通过Dataset和DataLoader组合使用实现

## 三个基本概念

### 批次 (batch)

一次喂入的一小组样本 (batch\_size 决定大小)

### 迭代 (iteration)

完成一个 batch 的前向+反向+更新

### 轮次 (epoch)

训练集全部样本过一遍

一个epoch包含的**迭代次数** =  $\lceil N / \text{batch\_size} \rceil$



# Step 1. 定制数据接口

- Dataset本身是一个抽象类，需要通过继承才能使用

## 定义Dataset子类FeatSet

- 读取每条影评样本的BoW特征和标签
- 稀疏存储的feats在喂给网络前要变为稠密数组 (L12)

| 方法                            | 功能描述                       |
|-------------------------------|----------------------------|
| <code>__init__</code>         | 指定数据列表，数据初始化，完成数据读取与基本处理   |
| <code>__len__</code>          | 返回数据集中样本总数                 |
| <code>__getitem__</code>      | 根据给定索引访问单个样本，用于逐个或按批次遍历数据集 |
| <code>transform</code>        | (可选) 对输入样本进行转换（如图像增强、归一化等） |
| <code>target_transform</code> | (可选) 对标签进行转换（如分类标签编码等）     |

```
1 from torch.utils.data import Dataset
2
3 class FeatSet(Dataset):
4     def __init__(self, feats, labels):
5         self.feats = feats
6         self.labels = labels
7
8     def __len__(self):
9         return len(self.labels)
10
11     def __getitem__(self, idx):
12         dense = self.feats[idx].toarray().squeeze()
13         return torch.tensor(dense, dtype=torch.float32), torch.tensor(self.labels[idx], dtype=torch.long)
```

# 使用数据接口

- 创建两个DataLoader对象，分别用于加载训练集和测试集

表 5.10: DataLoader常用初始化参数

| 参数                      | 默认值   | 功能描述                                                                          |
|-------------------------|-------|-------------------------------------------------------------------------------|
| <code>dataset</code>    | (必填)  | 数据源，即自定义的 <code>Dataset</code> 子类对象                                           |
| <code>batch_size</code> | 1     | 每个批次中包含的样本数量                                                                  |
| <code>shuffle</code>    | False | 是否以乱序读取数据， <code>train_loader</code> 设为True， <code>test_loader</code> 设为False |
| <code>drop_last</code>  | False | 当最后一个批次不足量时舍弃                                                                 |

数据预处理：

复用5.2.1节的代码加载imdb训练集和测试集，并基于训练集拟合Bow特征提取器

```

1 import pandas as pd
2 from sklearn.feature_extraction.text import CountVectorizer
3
4 train_file = './output/imdb_labelled_train.csv'
5 test_file = './output/imdb_labelled_test.csv'
6 df_train, df_test = pd.read_csv(train_file), pd.read_csv(test_file)
7 sents_train, sents_test = df_train['sentence'].values, df_test['sentence'].values
8 y_train, y_test = df_train['label'].values, df_test['label'].values
9 print(f'训练样本数量: {len(sents_train)}') # 800
10 print(f'测试样本数量: {len(sents_test)}') # 200
11
12 fe = CountVectorizer()
13 fe.fit(sents_train) # 使用训练数据构建Bow特征提取器
14 vob = fe.get_feature_names_out()
15 vocab_size = len(vob)
16 print(f'词表大小: {vocab_size}') # 词表大小: 2647

```

```

1 from torch.utils.data import DataLoader
2
3 train_ds = FeatSet(fe.transform(sents_train), y_train)
4 test_ds = FeatSet(fe.transform(sents_test), [0]*len(sents_test)) #推理阶段无真实标签可用
5
6 train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)
7 test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)
8
9 print(len(train_loader), len(test_loader)) # 13 7

```

! 测试集用全零标签，避免标签泄露



## Step 2. 搭建网络

表 5.11: Module类必要方法

| 方法                    | 功能描述                             |
|-----------------------|----------------------------------|
| <code>__init__</code> | 构造方法，用于定义网络结构，包括每层的类型和输入、输出维度    |
| <code>forward</code>  | 前向计算方法，给出前向计算流程，即数据如何从输入层逐步流向输出层 |

- 通过继承Module来实现
- 自定义SimpleNet：仅包含一个线性层的迷你神经网络
  - 线性层通过`nn.Linear(in_dim, out_dim)`创建，其作用是将一个（或一组）长度为`in_dim`的特征通过线性变换变为一个（或一组）长度为`out_dim`的特征

```
1 import torch.nn as nn
2
3 class SimpleNet(nn.Module):
4     def __init__(self, vocab_size, num_class=2):
5         super().__init__()
6         self.net = nn.Linear(vocab_size, num_class)
7
8     def forward(self, x):
9         return self.net(x)
```

`vocab_size`: Bow特征长度  
`num_class`: 类别数（正面评论/负面评论）



## Step 3. 训练模型

- 通过声明对象的方式设置各个训练选项

- L7: 损失函数
- L8: 待训练参数
- L9: 优化器
- L10: 训练轮次
- L12: 调度器

```
1 from transformers import get_linear_schedule_with_warmup
2
3 model = SimpleNet(vocab_size)
4 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('Using device:', device)
6 model.to(device) #模型 -> GPU
7 criterion = nn.CrossEntropyLoss() #设定损失函数
8 trainable_params = [p for p in model.parameters() if p.requires_grad]
9 optimizer = torch.optim.AdamW(trainable_params, lr=1e-4)
10 max_epoch = 300
11 total_steps = len(train_loader) * max_epoch
12 scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=total_steps)
13 print("可训练参数及参数量:")
14 print([(name, param.numel()) for name, param in model.named_parameters() if param.requires_grad])
```

Using device: cpu

可训练参数及参数量:

[('net.weight', 5294), ('net.bias', 2)]

vocab\_size为2647

线性变换矩阵包含 $2647 \times 2 = 5294$ 个参数

**!** 学习率 (lr) 过小影响训练效率、过大影响训练稳定性, 可通过调度器在训练过程中动态调整学习率初始值 (L9) 的设置仍高度依赖开发者的经验

# Step 3. 训练模型

## • 训练过程

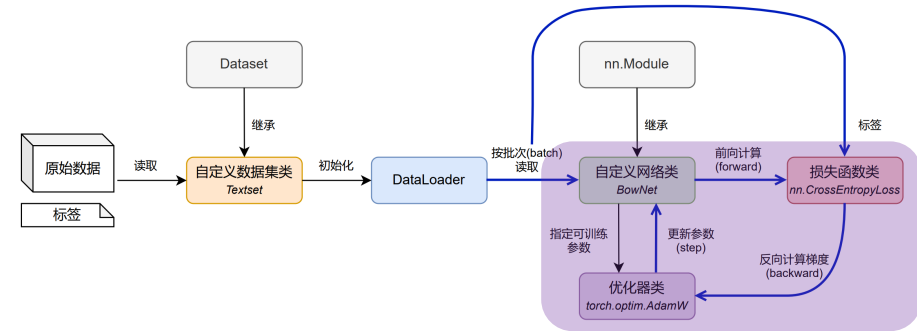
- 反向计算前 (L12) , 梯度先清零 (L11)

```

1 import time
2
3 t0 = time.perf_counter() # 记录开始时刻
4 for epoch in range(max_epoch):
5     model.train()
6     total_loss = 0
7     for batch in train_loader:
8         xb, yb = batch
9         xb, yb = xb.to(device), yb.to(device)
10        loss = criterion(model(xb), yb)
11        optimizer.zero_grad()
12        loss.backward()
13        optimizer.step()
14        scheduler.step()
15        total_loss += loss.item()
16    total_loss /= len(train_loader) # 注意要在内循环之外
17    if epoch % 20 == 0: # 每20轮次打印一次训练损失
18        print(f"epoch {epoch:02d} : Loss = {total_loss:.4f}")
19    if device.type == "cuda":
20        torch.cuda.synchronize(device)
21 total_time = time.perf_counter() - t0 # 训练结束时
22 print(f"Training finished in {total_time/60:.2f} min")

```

前向计算由哪条语句完成?



```

epoch 00 : Loss = 0.6942
epoch 20 : Loss = 0.6440
epoch 40 : Loss = 0.6047
epoch 60 : Loss = 0.5717
epoch 80 : Loss = 0.5441
epoch 100 : Loss = 0.5204
epoch 120 : Loss = 0.5019
epoch 140 : Loss = 0.4870
epoch 160 : Loss = 0.4742
epoch 180 : Loss = 0.4611
epoch 200 : Loss = 0.4521
epoch 220 : Loss = 0.4448
epoch 240 : Loss = 0.4384
epoch 260 : Loss = 0.4337
epoch 280 : Loss = 0.4327
Training finished in 0.07 min

```



# Step 4. 模型推理 & Step 5. 性能评估

- 需先通过`model.eval()`将模型设置为推理模式

```
1 from sklearn.metrics import classification_report
2
3 model.eval() # 将模型设置为推理模式
4 all_preds = []
5 with torch.no_grad(): # 禁用梯度计算, 提高推理效率
6     for xb, _ in test_loader:
7         xb = xb.to(device)
8         output = model(xb)
9         preds = torch.argmax(output, dim=1).cpu().numpy() # 每个样本取得分最高的类别作为预测结果
10        all_preds.extend(preds)
11 # 性能评测
12 print(classification_report(y_test, all_preds, digits=3))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.832     | 0.848  | 0.840    | 105     |
| 1            | 0.828     | 0.811  | 0.819    | 95      |
| accuracy     |           |        | 0.830    | 200     |
| macro avg    | 0.830     | 0.829  | 0.829    | 200     |
| weighted avg | 0.830     | 0.830  | 0.830    | 200     |

# 用额外样本测试SimpleNet

- 复用示例代码5.8的测试样本

```
1 input_texts = ['this is a good movie', 'the movie is horrible', 'a horrible movie', 'not a bad movie']
2 input_ds = FeatSet(fe.transform(input_texts), [0]*len(input_texts))
3 input_loader = DataLoader(input_ds, batch_size=16, shuffle=False)
4 model.eval()
5 all_scores = []
6 with torch.no_grad():
7     for batch in input_loader:
8         xb, yb = batch
9         xb = xb.to(device)
10        yb = yb.to(device)
11        output = model(xb)
12        scores = torch.softmax(output, dim=1).cpu().numpy()
13        all_scores.append(scores)
14 scores_simplenet = np.vstack(all_scores)
15 print(scores_simplenet)
```

```
[[0.4928521  0.50714785]
 [0.5756278  0.42437217]
 [0.5802901  0.41970998]
 [0.6585362  0.3414638 ]]
```

“not a bad movie” 依旧被识别为**负面**

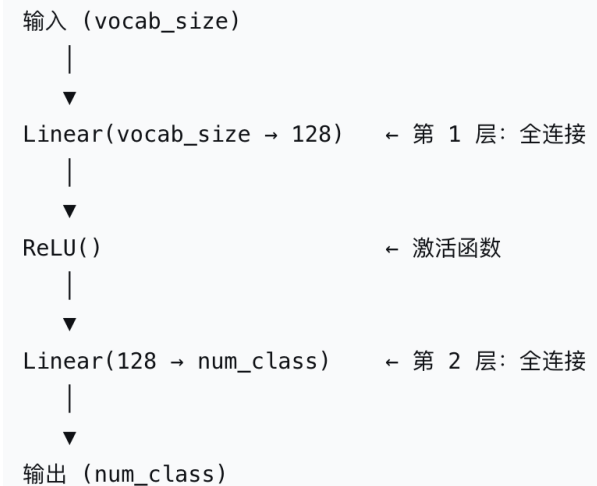
特征固有局限性能否通过增强网络学习能力来弥补？

## 课堂练习-5-2-2

- 尝试往SimpleNet中加入一个中间层，变成一个更复杂的双层神经网络

```
self.net = nn.Sequential(  
    nn.Linear(vocab_size, 128),  
    nn.ReLU(),  
    nn.Linear(128, num_class))
```

在相同训练配置下，对比单层与双层网络的性能





## 案例2：基于BERT的英文影评情感识别

- 基于预训练的BERT构造更强的影评情感识别模型
  - BERT由谷歌研发，基于Transformer架构的深度学习神经网络语言模型
- 熟悉基于预训练模型的深度学习 workflow



# Step 1. 下载BERT模型

- 利用huggingface\_hub从Hugging Face (HF) 下载
  - HF是当前各种AI模型的网上集散地

```
1 from huggingface_hub import snapshot_download
2 snapshot_download(repo_id="google-bert/bert-base-uncased", local_dir="bert-base-uncased", endpoint="https://hf-mirror.com")
```

Fetching 16 files: 100% ██████████ 16/16 [00:28<00:00, 2.04s/it]

README.md: █████ 10.5k/? [00:00<00:00, 920kB/s]

flax\_model.msgpack: 100% ██████████ 438M/438M [00:23<00:00, 16.0MB/s]

weight.bin: 100% ██████████ 532M/532M [00:25<00:00, 22.4MB/s]

.gitattributes: █████ 491/? [00:00<00:00, 32.1kB/s]

config.json: █████ 570/? [00:00<00:00, 43.4kB/s]

model.mlmodel: 100% ██████████ 165k/165k [00:00<00:00, 411kB/s]

Manifest.json: █████ 617/? [00:00<00:00, 84.3kB/s]

model.onnx: 100% ██████████ 532M/532M [00:24<00:00, 24.2MB/s]

model.safetensors: 100% ██████████ 440M/440M [00:23<00:00, 20.9MB/s]

pytorch\_model.bin: 100% ██████████ 440M/440M [00:24<00:00, 23.5MB/s]

LICENSE: █████ 11.4k/? [00:00<00:00, 725kB/s]

rust\_model.ot: 100% ██████████ 534M/534M [00:26<00:00, 42.2MB/s]

tf\_model.h5: 100% ██████████ 536M/536M [00:25<00:00, 39.8MB/s]

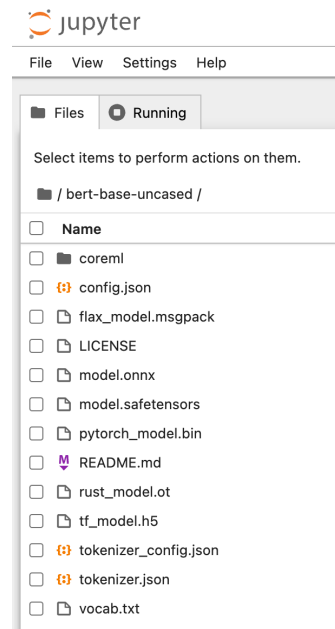
tokenizer.json: █████ 466k/? [00:00<00:00, 474kB/s]

tokenizer\_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 3.04kB/s]

vocab.txt: █████ 232k/? [00:00<00:00, 334kB/s]



下载到当前目录下bert-base-uncased文件夹中



# 使用BERT模型

- 尝试加载模型，以确保模型已下载成功

```
1 from transformers import BertTokenizerFast, BertForSequenceClassification
2
3 local_model_path = './bert-base-uncased'
4 model = BertForSequenceClassification.from_pretrained(local_model_path, num_labels=2)
5 num_params = sum([param.numel() for name, param in model.named_parameters() if param.requires_grad])
6 print(f"可训练参数量: {num_params}")
7 print("分类层:", model.classifier)
8 tokenizer = BertTokenizerFast.from_pretrained(local_model_path)
```

BertForSequenceClassification: **transformers**库提供的以BERT为内核的文本分类器

可训练参数量: 109483778 (**1亿!**)

分类层: Linear(in\_features=768, out\_features=2, bias=True)

第8行加载的是BERT专用的分词器，其作用是将输入文本编码为数值化的token ID序列

- 定制数据接口时必须使用该分词器

# 使用BERT模型

- 理解BERT分词器的输入、输出
  - 把文本（词序列）编码为 token ID 序列
  - 填充符：[CLS] 句首 [SEP] 句尾 [PAD] 填充
  - 注意力掩码（attention\_mask）
    - 1表示真实词，参与计算
    - 0表示填充符，不参与计算

每个输入句子都会被添加[CLS]和[SEP]，因此最多保留前`max_length-2`个词

```
1 text = ["a good movie", "a bad movie", "This brilliant film deeply moved me with stunning visuals"]
2 encoded = tokenizer(text, add_special_tokens=True, max_length=8, truncation=True, padding='max_length', return_tensors='pt')
3 decoded = tokenizer.batch_decode(encoded['input_ids'])
4 print(encoded)
5 print(decoded)
```

```
{'input_ids': tensor([[ 101, 1037, 2204, 3185,  102,    0,    0,    0],
                      [ 101, 1037, 2919, 3185,  102,    0,    0,    0],
                      [ 101, 2023, 8235, 2143, 6171, 2333, 2033,  102]]), 'token_type_ids': tensor([[0, 0, 0, 0, 0, 0, 0, 0],
                      [0, 0, 0, 0, 0, 0, 0, 0],
                      [0, 0, 0, 0, 0, 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 0, 0, 0],
                      [1, 1, 1, 1, 1, 0, 0, 0],
                      [1, 1, 1, 1, 1, 1, 1, 1]])}
['[CLS] a good movie [SEP] [PAD] [PAD] [PAD]', '[CLS] a bad movie [SEP] [PAD] [PAD] [PAD]', '[CLS] this brilliant film deeply moved me [SEP] [PAD]']
```

模型推理时需要的字段

- input\_ids
- attention\_mask

## Step 2. 定制数据接口

- 定义面向BERT的数据集类 `BertDataset`

```
1 class BertDataset(Dataset):
2     def __init__(self, sents, labels, tokenizer, max_len=64):
3         self.texts = sents
4         self.labels = labels
5         self.tokenizer = tokenizer
6         self.max_len = max_len
7
8     def __len__(self):
9         return len(self.labels)
10
11    def __getitem__(self, idx):
12        encoded = self.tokenizer(
13            self.texts[idx],
14            add_special_tokens=True,
15            truncation=True,
16            padding='max_length',
17            max_length=self.max_len,
18            return_tensors='pt'
19        )
20        item = {key: val.squeeze(0) for key, val in encoded.items()}
21        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
22        return item
```

L5: 通过初始化方法将BERT  
分词器设置为类属性  
L12: 读取文本数据时使用  
分词器进行编码

# 使用数据接口

- 创建一个DataLoader对象，用于加载训练集

```
1 train_ds = BertDataset(sents_train, y_train, tokenizer)
2 train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
3 print(f'Training data set: {len(train_ds)}')
4 print(f'Number of training batches: {len(train_loader)}')
```

Training data set: 800

Number of training batches: 25

对比之前的面向Bow特征的数据接口，可见二者的代码结构高度相似

```
train_ds = FeatSet(fe.transform(sents_train), y_train)
train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)
```



## Step 3. 微调BERT分类器

- 设定训练选项

```
1 model.to(device)
2 optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)
3 max_epoch = 10
4 total_steps = len(train_loader) * max_epoch
5 scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=total_steps)
```

相比SimpleNet的从零开始训练方案

- 微调使用的学习率更低  $lr=5e-5$
- 训练轮次更少  $max\_epoch=10$

## Step 3. 微调BERT分类器

- 整体框架与训练SimpleNet类似

```
1 from tqdm import tqdm
2
3 t0 = time.perf_counter()
4 for epoch in range(max_epoch):
5     model.train()
6     total_loss = 0
7     for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}"):
8         batch = {k: v.to(device) for k, v in batch.items()}
9         outputs = model(**batch)
10        loss = outputs.loss
11        optimizer.zero_grad()
12        loss.backward()
13        optimizer.step()
14        scheduler.step()
15        total_loss += loss.item()
16    print(f"Epoch {epoch+1}: Loss = {total_loss / len(train_loader):.4f}")
17    if device.type == "cuda":
18        torch.cuda.synchronize(device)
19    total_time = time.perf_counter() - t0
20    print(f"Training finished in {total_time/60:.2f} min")
```

### 主要差别

- 前向计算 (L9) : \*\*batch将 batch从字典格式拆成关键字实参
- 进度条 (L7) : 让读者更好得了解训练进度



# Step 4. 模型推理 & Step 5. 性能评估

- 仍需先通过`model.eval()`将模型设置为推理模式

```
1 test_ds = BertDataset(sents_test, [0]*len(sents_test), tokenizer) #推理阶段无真实标签可用
2 test_loader = DataLoader(test_ds, batch_size=16, shuffle=False)
3 model.eval()
4 all_preds = []
5 with torch.no_grad():
6     for batch in test_loader:
7         batch = {k: v.to(device) for k, v in batch.items()}
8         input_ids = batch['input_ids']
9         attention_mask = batch['attention_mask']
10        labels = batch['labels']
11        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
12        preds = torch.argmax(outputs.logits, dim=1).cpu().numpy()
13        all_preds.extend(preds)
14 print(classification_report(y_test, all_preds, digits=3))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.917     | 0.952  | 0.935    | 105     |
| 1            | 0.945     | 0.905  | 0.925    | 95      |
| accuracy     |           |        | 0.930    | 200     |
| macro avg    | 0.931     | 0.929  | 0.930    | 200     |
| weighted avg | 0.931     | 0.930  | 0.930    | 200     |



# 用额外样本测试BERT分类器

- SimpleNet出错的例子已能正确识别

```
1 input_texts = ['this is a good movie', 'the movie is horrible', 'a horrible movie', 'not a bad movie']
2 input_ds = BertDataset(input_texts, [0]*len(input_texts), tokenizer)
3 input_loader = DataLoader(input_ds, batch_size=16, shuffle=False)
4 model.eval()
5 all_scores = []
6 with torch.no_grad():
7     for batch in input_loader:
8         batch = {k: v.to(device) for k, v in batch.items()}
9         input_ids = batch['input_ids']
10        attention_mask = batch['attention_mask']
11        labels = batch['labels']
12        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
13        scores = torch.softmax(outputs.logits, dim=1).cpu().numpy()
14        all_scores.append(scores)
15 scores_bert = np.vstack(all_scores)
16 print(scores_bert)
```

```
[[5.1692297e-04 9.9948299e-01]
 [9.9925023e-01 7.4973638e-04]
 [9.9897802e-01 1.0219773e-03]
 [1.3519935e-03 9.9864799e-01]]
```

| review                | SimpleNet | BERT     |
|-----------------------|-----------|----------|
| this is a good movie  | 0.507148  | 0.999483 |
| the movie is horrible | 0.424372  | 0.000750 |
| a horrible movie      | 0.419710  | 0.001022 |
| not a bad movie       | 0.341464  | 0.998648 |

## 案例3：基于M3E的中文酒店评论情感识别

- 基于M3E构造（比5.2.2节）更强的酒店评论情感识别模型
  - M3E由月之暗面研发，侧重中文的基座模型（5.4节）
- 熟悉基于基座模型的深度学习 workflows





# Step 1. 下载M3E模型

- 利用huggingface\_hub从Hugging Face (HF) 下载
  - 迷你版 (0.24亿参数) : m3e-small

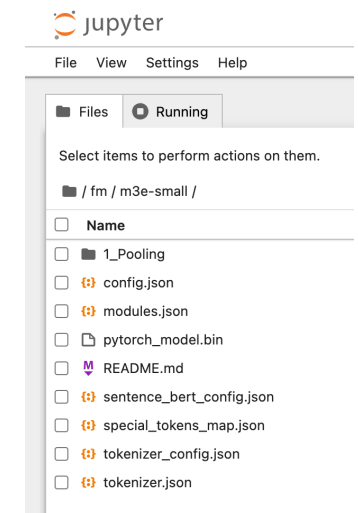
下载到当前目录下fm/m3e-small文件夹中

```
1 from huggingface_hub import snapshot_download
2
3 endpoint = 'https://hf-mirror.com'
4 model_list = [("sentence-transformers/all-MiniLM-L6-v2", "fm/all-MiniLM-L6-v2"),
5               ("moka-ai/m3e-small", "fm/m3e-small"),
6               ("sentence-transformers/clip-ViT-B-32", "fm/clip-ViT-B-32")]
7 for repo_id, local_dir in model_list:
8     snapshot_download(repo_id=repo_id, local_dir=local_dir, endpoint=endpoint)
```

Fetching 30 files: 100%  30/30 [00:00<00:00, 523.01it/s]

Fetching 10 files: 100%  10/10 [00:00<00:00, 963.19it/s]

Fetching 12 files: 100%  12/12 [00:00<00:00, 827.52it/s]





# 加载M3E模型

- 使用第三方库sentence\_transformers

```
import sentence_transformers as sentrans
```

```
1 model = sentrans.SentenceTransformer("./fm/m3e-small")
2 # 输出模型的基本信息
3 p = sum(p.numel() for p in model.parameters())
4 vocab_size = model.tokenizer.vocab_size
5 max_len = model.get_max_seq_length()
6 #d = model.get_sentence_embedding_dimension()
7 d = model.get_embedding_dimension()
8 print("总参数量", f"{p/1e6:.1f} M")
9 print("词典大小", vocab_size)
10 print("嵌入维度", d)
11 print("输入文本最大长度", max_len)
```

总参数量 24.0 M  
词典大小 21128  
嵌入维度 512  
输入文本最大长度 512



## Step 2. 文本嵌入

- 嵌入 (embedding)：由基座模型提取的特征

数据预处理：

复用5.2.2节的代码加载酒店评论训练集和测试集

```
1 train_file = './output/hotel_labelled_train.csv'
2 test_file = './output/hotel_labelled_test.csv'
3 df_train, df_test = pd.read_csv(train_file), pd.read_csv(test_file)
4 sents_train, sents_test = df_train['review'].values, df_test['review'].values
5 y_train, y_test = df_train['label'].values, df_test['label'].values
6
7 print(f'训练样本数量: {len(sents_train)}') # 6212
8 print(f'测试样本数量: {len(sents_test)}') # 1553
```

训练样本数量: 6212

测试样本数量: 1553

将评论表示为M3E嵌入特征

```
1 x_train = model.encode(sents_train) #ZSR不需要训练样本, 故该语句在ZSR时可忽略
2 x_test = model.encode(sents_test)
3
4 print(f'训练集特征向量尺寸: {x_train.shape}') # 6212*512
5 print(f'测试集特征向量尺寸: {x_test.shape}') # 1553*512
```

训练集特征向量尺寸: (6212, 512)

测试集特征向量尺寸: (1553, 512)

## Step 3. 零样本识别 (ZSR)

- 不需要训练样本也可进行识别
- 原理：用M3E提取所有待识别标签的嵌入特征，再根据测试样本与标签之间的嵌入特征相似度判断该样本的类别

```
1 label_text = ['负面酒店评论是指顾客在入住体验后，对酒店服务、设施、卫生、环境等方面表达不满或失望的评论。  
这类评论通常包含对具体问题的描述，如房间脏乱、服务态度差、噪音扰民、设施故障、价格与体验不符等，旨在提醒其他潜在住客或促使酒店改进。', '正面酒店评论是顾客在入住后对酒店表示满意或赞赏的评价。这类评论通常提及服务热情周到、房间整洁舒适、设施齐全先进、位置便利、性价比高或超出预期等优点，旨在推荐给其他潜在住客并肯定酒店的表现。']  
2 label_emb = model.encode(label_text)  
3 scores = sentrans.util.cos_sim(x_test, label_emb)  
4 pred = np.argmax(scores, axis=1)  
5 pred = pred.cpu().numpy()  
6 report = classification_report(y_test, pred, digits=3)  
7 print(report)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.595     | 0.941  | 0.729    | 476     |
| 1            | 0.965     | 0.717  | 0.823    | 1077    |
| accuracy     |           |        | 0.786    | 1553    |
| macro avg    | 0.780     | 0.829  | 0.776    | 1553    |
| weighted avg | 0.852     | 0.786  | 0.794    | 1553    |

ZSR更倾向预测负面



# Step 4. 机器学习 + Step 5. 性能评估

- MSE嵌入特征 + LRC分类器

```
1 clf = LogisticRegression(max_iter=1000, C=0.01)
2 clf.fit(x_train, y_train)
3 pred = clf.predict(x_test)
4 report = classification_report(y_test, pred, digits=3)
5 print(report)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.870     | 0.813  | 0.840    | 476     |
| 1            | 0.920     | 0.946  | 0.933    | 1077    |
| accuracy     |           |        | 0.905    | 1553    |
| macro avg    | 0.895     | 0.880  | 0.887    | 1553    |
| weighted avg | 0.904     | 0.905  | 0.904    | 1553    |

在强特征的加持下，简单的分类器也能取得很好的分类效果



## 课堂练习-5-2-3

- 构建基于M3E的英文影评情感识别模型
  - ZSR方案
  - LRC方案